

## Arrays:

1. Develop a Java program that declares and initializes an array of integers. Print the elements of the array in reverse order.

```
public class workshop3 {  
    Run | Debug  
    public static void main(String[] args) {  
        int[] numbers = {1, 2, 3, 4, 5};  
  
        System.out.println(x:"Array elements in reverse order:");  
        for (int i = numbers.length - 1; i >= 0; i--) {  
            System.out.print(numbers[i] + " ");  
        }  
    }  
}
```

```
Array elements in reverse order:
```

```
5 4 3 2 1
```

```
PS C:\Users\sigma\OneDrive\Desktop\object oriented> █
```

2. Implement a Java program that finds the sum and average of elements in an array of floating-point numbers.

```
public class workshop3 {  
    Run | Debug  
    public static void main(String[] args) {  
        double[] numbers = {1.5, 2.5, 3.5, 4.5, 5.5};  
  
        double sum = 0;  
        for (double num : numbers) {  
            sum += num;  
        }  
  
        double average = sum / numbers.length;  
  
        System.out.println("Sum of elements: " + sum);  
        System.out.println("Average of elements: " + average);  
    }  
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\object oriented> java workshop3.java  
Sum of elements: 17.5  
Average of elements: 3.5
```

3. Write a Java program that checks if a given element is present in an array of strings. If present, print its index; otherwise, print a message indicating its absence.

```
public class workshop3 {  
    Run | Debug  
    public static void main(String[] args) {  
        String[] array = {"apple", "banana", "orange", "grape", "mango"};  
  
        String element = "orange";  
  
        int index = searchElement(array, element);  
  
        if (index != -1) {  
            System.out.println("Element '" + element + "' found at index: " + index);  
        } else {  
            System.out.println("Element '" + element + "' not found in the array.");  
        }  
    }  
  
    public static int searchElement(String[] array, String element) {  
        for (int i = 0; i < array.length; i++) {  
            if (array[i].equals(element)) {  
                return i;  
            }  
        }  
        return -1;  
    }  
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop3.java  
Element 'orange' found at index: 2
```

4. Create a Java program that sorts an array of integers in ascending order using the bubble sort algorithm.

```
public class workshop3 {  
    Run | Debug  
    public static void main(String[] args) {  
        int[] array = {64, 34, 25, 12, 22, 11, 90};  
  
        System.out.println(x:"Original array:");  
        printArray(array);  
  
        bubbleSort(array);  
  
        System.out.println(x:"\nSorted array:");  
        printArray(array);  
    }  
  
    public static void bubbleSort(int[] array) {  
        int n = array.length;  
        for (int i = 0; i < n-1; i++) {  
            for (int j = 0; j < n-i-1; j++) {  
                if (array[j] > array[j+1]) {  
                    int temp = array[j];  
                    array[j] = array[j+1];  
                    array[j+1] = temp;  
                }  
            }  
        }  
    }  
  
    public static void printArray(int[] array) {  
        for (int i = 0; i < array.length; i++) {  
            System.out.print(array[i] + " ");  
        }  
        System.out.println();  
    }  
}
```

```
11 12 22 25 34 64 90
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> 
```

## Basics of Classes and Objects:

5. Define a class named BankAccount with attributes accountNumber, balance, and accountHolderName, accountHolderAddress.

```
public class workshop3 {  
    private String accountNumber;  
    private double balance;  
    private String accountHolderName;  
    private String accountHolderAddress;  
  
    public workshop3(String accountNumber, double balance, String accountHolderName, String accountHolderAddress) {  
        this.accountNumber = accountNumber;  
        this.balance = balance;  
        this.accountHolderName = accountHolderName;  
        this.accountHolderAddress = accountHolderAddress;  
    }  
  
    public String getAccountNumber() {  
        return accountNumber;  
    }  
  
    public void setAccountNumber(String accountNumber) {  
        this.accountNumber = accountNumber;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void setBalance(double balance) {  
        this.balance = balance;  
    }  
  
    public String getAccountHolderName() {  
        return accountHolderName;  
    }  
  
    public void setAccountHolderName(String accountHolderName) {  
        this.accountHolderName = accountHolderName;  
    }  
}
```

6. Create an object of this class and initialize its attributes.

```
public class Main {  
    public static void main(String[] args) {  
        BankAccount account = new BankAccount("1234567890", 1000.00, "Bibek", "ktm");  
  
        System.out.println("Initial Account Information:");  
        account.displayAccountInfo();  
  
        account.deposit(500.50);  
  
        account.withdraw(200.75);  
  
        System.out.println("\nUpdated Account Information:");  
        account.displayAccountInfo();  
    }  
}
```

**Methods:**

7. Create a method, **depositMoney()** in the BankAccount class to deposit money. Implement another method, **withdrawMoney()** to withdraw money. (The current balance should also be printed).

```
public class workshop3 {  
    public static class BankAccount {  
  
        public void depositMoney(double amount) {  
            if (amount > 0) {  
                this.balance += amount;  
                System.out.println("Deposited $" + amount + ". Current balance: $" + this.balance);  
            } else {  
                System.out.println(x:"Invalid amount to deposit.");  
            }  
        }  
  
        public void withdrawMoney(double amount) {  
            if (amount > 0 && amount <= this.balance) {  
                this.balance -= amount;  
                System.out.println("Withdrew $" + amount + ". Current balance: $" + this.balance);  
            } else {  
                System.out.println(x:"Insufficient funds or invalid amount to withdraw.");  
            }  
        }  
    }  
}  
  
Run | Debug  
public static void main(String[] args) {  
    BankAccount account = new BankAccount(initialBalance:1000);  
    account.depositMoney(amount:50067);  
    account.withdrawMoney(amount:20450);  
    account.withdrawMoney(amount:189500);  
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop3.java  
Deposited $50067.0. Current balance: $51067.0  
Withdrew $20450.0. Current balance: $30617.0  
Insufficient funds or invalid amount to withdraw.
```

8. Create a class Lamp with attributes **isOn** to store boolean value. Also create a method **turnOn()** to turn on the light, and **turnOff()** to turn off the light and print the on status of the light.

```
public class workshop3 {  
    public static class Lamp {  
        private boolean isOn;  
  
        // Constructor  
        public Lamp() {  
            this.isOn = false;  
        }  
  
        public void turnOn() {  
            this.isOn = true;  
            System.out.println(x:"The lamp is now turned on.");  
        }  
  
        public void turnOff() {  
            this.isOn = false;  
            System.out.println(x:"The lamp is now turned off.");  
        }  
  
        public boolean isLampOn() {  
            return this.isOn;  
        }  
    }  
}  
  
Run | Debug  
public static void main(String[] args) {  
    Lamp lamp = new Lamp();  
  
    System.out.println("Is the lamp on? " + lamp.isLampOn());  
  
    lamp.turnOn();  
}
```

```
    public void turnOff() {
        this.isOn = false;
        System.out.println(x:"The lamp is now turned off.");
    }

    public boolean isLampOn() {
        return this.isOn;
    }
}

Run | Debug
public static void main(String[] args) {
    Lamp lamp = new Lamp();

    System.out.println("Is the lamp on? " + lamp.isLampOn());

    lamp.turnOn();
    System.out.println("Is the lamp on? " + lamp.isLampOn());

    lamp.turnOff();
    System.out.println("Is the lamp on? " + lamp.isLampOn());
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop3.java
Is the lamp on? false
The lamp is now turned on.
Is the lamp on? true
The lamp is now turned off.
Is the lamp on? false
PS C:\Users\sigma\OneDrive\Desktop\java> 
```

## Constructors:



9. Implement a parameterized constructor for the BankAccount class that initializes the account attributes. Create an object using this constructor.

```
public class workshop3 {  
    public static class BankAccount {  
        private double balance;  
  
        public BankAccount(double initialBalance) {  
            this.balance = initialBalance;  
        }  
  
        public void depositMoney(double amount) {  
            if (amount > 0) {  
                this.balance += amount;  
                System.out.println("Deposited $" + amount + ". Current balance: $" + this.balance);  
            } else {  
                System.out.println(x:"Invalid amount to deposit.");  
            }  
        }  
  
        public void withdrawMoney(double amount) {  
            if (amount > 0 && amount <= this.balance) {  
                this.balance -= amount;  
                System.out.println("Withdrew $" + amount + ". Current balance: $" + this.balance);  
            } else {  
                System.out.println(x:"Insufficient funds or invalid amount to withdraw.");  
            }  
        }  
    }  
}  
  
Run | Debug  
public static void main(String[] args) {  
    BankAccount account = new BankAccount(initialBalance:1500);  
    account.depositMoney(amount:500);  
    account.withdrawMoney(amount:2000);  
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop3.java  
Deposited $7500.0. Current balance: $79000.0  
Insufficient funds or invalid amount to withdraw.  
PS C:\Users\sigma\OneDrive\Desktop\java>
```

10. Implement a no-argument constructor that prints out “**User created!**” as soon as the instance of the user is created.

```
public class workshop3 {  
    public static class BankAccount {  
        private double balance;  
  
        public BankAccount() {  
            System.out.println(x:"User created!");  
        }  
  
        public BankAccount(double initialBalance) {  
            this.balance = initialBalance;  
        }  
  
        public void depositMoney(double amount) {  
            if (amount > 0) {  
                this.balance += amount;  
                System.out.println("Deposited $" + amount + ". Current balance: $" + this.balance);  
            } else {  
                System.out.println(x:"Invalid amount to deposit.");  
            }  
        }  
  
        public void withdrawMoney(double amount) {  
            if (amount > 0 && amount <= this.balance) {  
                this.balance -= amount;  
                System.out.println("Withdrew $" + amount + ". Current balance: $" + this.balance);  
            } else {  
                System.out.println(x:"Insufficient funds or invalid amount to withdraw.");  
            }  
        }  
    }  
}
```

Run | Debug

```
public static void main(String[] args) {  
    BankAccount account1 = new BankAccount();  
    account1.depositMoney(amount:50090);  
  
    BankAccount account2 = new BankAccount(initialBalance:1000);  
    account2.withdrawMoney(amount:28700);  
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop3.java
User created!
Deposited $50090.0. Current balance: $50090.0
Insufficient funds or invalid amount to withdraw.
PS C:\Users\sigma\OneDrive\Desktop\java> 
```

## Constructor Overloading:

11. Create a class named ,"**Box**" with attributes **width**, **height**, and **depth**. Create multiple constructors for handling following object declarations. Also declare a method **getVolume()** that prints the volume of the declared:
  - a. For a cube, declare a constructor to take length only.
  - b. For a cuboid, declare a constructor to take length, breadth, and height.
  - c. For no parameter, declare a no-argument constructor that sets **length = 10**, **breadth = 8**, and **height = 12**.

```
public class workshop3 {  
    public static class Box {  
        private double width;  
        private double height;  
        private double depth;  
  
        public Box(double length) {  
            this.width = length;  
            this.height = length;  
            this.depth = length;  
        }  
  
        public Box(double length, double breadth, double height) {  
            this.width = length;  
            this.height = breadth;  
            this.depth = height;  
        }  
  
        public Box() {  
            this.width = 10;  
            this.height = 8;  
            this.depth = 12;  
        }  
  
        public void getVolume() {  
            double volume = width * height * depth;  
            System.out.println("Volume of the box: " + volume);  
        }  
  
        Run | Debug  
        public static void main(String[] args) {
```

```
}  
  
Run | Debug  
public static void main(String[] args) {  
    Box cube = new Box(length:5);  
    cube.getVolume();  
  
    Box cuboid = new Box(length:5, breadth:4, height:3);  
    cuboid.getVolume();  
  
    Box defaultBox = new Box();  
    defaultBox.getVolume();  
}  
}
```

```
Volume of the box: 125.0  
Volume of the box: 60.0  
Volume of the box: 960.0  
PS C:\Users\sigma\OneDrive\De
```

## Access Modifiers:

12. Set the balance attribute in the BankAccount class as private.  
Provide public getter methods for the balance.

```
public class workshop3 {  
    public static class BankAccount {  
        private double balance;  
  
        public BankAccount(double initialBalance) {  
            this.balance = initialBalance;  
        }  
  
        public void depositMoney(double amount) {  
            if (amount > 0) {  
                this.balance += amount;  
                System.out.println("Deposited $" + amount + ". Current balance: $" + this.balance);  
            } else {  
                System.out.println(x:"Invalid amount to deposit.");  
            }  
        }  
  
        public void withdrawMoney(double amount) {  
            if (amount > 0 && amount <= this.balance) {  
                this.balance -= amount;  
                System.out.println("Withdrew $" + amount + ". Current balance: $" + this.balance);  
            } else {  
                System.out.println(x:"Insufficient funds or invalid amount to withdraw.");  
            }  
        }  
    }  
  
    // Getter method for balance  
    public double getBalance() {  
        return this.balance;  
    }  
}
```

```
// Getter method for balance  
public double getBalance() {  
    return this.balance;  
}  
  
Run | Debug  
public static void main(String[] args) {  
    BankAccount account = new BankAccount(initialBalance:1000);  
    account.depositMoney(amount:500);  
    account.withdrawMoney(amount:200);  
    System.out.println("Current balance: $" + account.getBalance());  
}
```

```
Deposited $500.0. Current balance: $1500.0  
Withdrew $200.0. Current balance: $1300.0  
Current balance: $1300.0
```

## **Encapsulation:**

13. Create a class Address with private attributes street, city, and zipCode. Use encapsulation and provide getter and setter methods.

```
public class workshop3 {  
    public static class Address {  
        private String street;  
        private String city;  
        private String zipCode;  
  
        public String getStreet() {  
            return street;  
        }  
  
        public void setStreet(String street) {  
            this.street = street;  
        }  
  
        public String getCity() {  
            return city;  
        }  
  
        public void setCity(String city) {  
            this.city = city;  
        }  
  
        public String getZipCode() {  
            return zipCode;  
        }  
  
        public void setZipCode(String zipCode) {  
            this.zipCode = zipCode;  
        }  
    }  
}
```



```
}  
  
Run | Debug  
public static void main(String[] args) {  
    Address address = new Address();  
    address.setStreet(street:"ktm");  
    address.setCity(city:"ktm");  
    address.setZipCode(zipCode:"123453456");  
  
    System.out.println("Street: " + address.getStreet());  
    System.out.println("City: " + address.getCity());  
    System.out.println("Zip Code: " + address.getZipCode());  
}  
}
```

```
Street: ktm  
City: ktm  
Zip Code: 123453456  
PS C:\Users\sigma\OneDrive\Desktop\java>
```

## Combining Concepts:

14. Create a class Customer with private attributes customerId, name, and a BankAccount attribute. Implement a parameterized constructor and encapsulate the attributes. Provide getter method. Instantiate multiple Customer objects with different values and demonstrate the use of getters and setters.

```
public class workshop3 {  
    public static class BankAccount {  
        private double balance;  
  
        public BankAccount(double initialBalance) {  
            this.balance = initialBalance;  
        }  
  
        public void depositMoney(double amount) {  
            if (amount > 0) {  
                this.balance += amount;  
                System.out.println("Deposited $" + amount + ". Current balance: $" + this.balance);  
            } else {  
                System.out.println(x:"Invalid amount to deposit.");  
            }  
        }  
  
        public void withdrawMoney(double amount) {  
            if (amount > 0 && amount <= this.balance) {  
                this.balance -= amount;  
                System.out.println("Withdrew $" + amount + ". Current balance: $" + this.balance);  
            } else {  
                System.out.println(x:"Insufficient funds or invalid amount to withdraw.");  
            }  
        }  
  
        public double getBalance() {  
            return this.balance;  
        }  
    }  
}
```

```
public static class Customer {
    private int customerId;
    private String name;
    private BankAccount bankAccount;

    public Customer(int customerId, String name, BankAccount bankAccount) {
        this.customerId = customerId;
        this.name = name;
        this.bankAccount = bankAccount;
    }

    public int getCustomerId() {
        return customerId;
    }

    public String getName() {
        return name;
    }

    public BankAccount getBankAccount() {
        return bankAccount;
    }
}

Run | Debug
public static void main(String[] args) {
    BankAccount account1 = new BankAccount(initialBalance:1000);
    BankAccount account2 = new BankAccount(initialBalance:2000);

    Customer customer1 = new Customer(customerId:1, name:"Bibeksh", account1);
    Customer customer2 = new Customer(customerId:2, name:"amrit", account2);
}
```

```
Run | Debug
public static void main(String[] args) {
    BankAccount account1 = new BankAccount(initialBalance:1000);
    BankAccount account2 = new BankAccount(initialBalance:2000);

    Customer customer1 = new Customer(customerId:1, name:"Bibeksh", account1);
    Customer customer2 = new Customer(customerId:2, name:"amrit", account2);

    System.out.println("Customer 1 ID: " + customer1.getCustomerId());
    System.out.println("Customer 1 Name: " + customer1.getName());
    System.out.println("Customer 1 Account Balance: $" + customer1.getBankAccount().getBalance());

    System.out.println("Customer 2 ID: " + customer2.getCustomerId());
    System.out.println("Customer 2 Name: " + customer2.getName());
    System.out.println("Customer 2 Account Balance: $" + customer2.getBankAccount().getBalance());
}
}
```

```
Customer 1 ID: 1
Customer 1 Name: Bibeksh
Customer 1 Account Balance: $1000.0
Customer 2 ID: 2
Customer 2 Name: amrit
Customer 2 Account Balance: $2000.0
PS C:\Users\sigma\OneDrive\Desktop\java>
```

## Constructors Overloading:

15. Implement multiple constructors for the BankAccount class with different parameter sets. Use constructor overloading to create objects with different initialization scenarios.

Introduction to Object Oriented Programming  
Workshop 3

```
public class BankAccount {
    private String accountNumber;
    private double balance;
    private String accountHolderName;
    private String accountHolderAddress;

    public BankAccount(String accountNumber, double balance, String accountHolderName, String accountHolderAddress) {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.accountHolderName = accountHolderName;
        this.accountHolderAddress = accountHolderAddress;
    }

    public BankAccount(String accountNumber, String accountHolderName, String accountHolderAddress) {
        this(accountNumber, 0.0, accountHolderName, accountHolderAddress);
    }

    public BankAccount(String accountNumber, double balance, String accountHolderName) {
        this(accountNumber, balance, accountHolderName, "");
    }

    public BankAccount(String accountNumber, double balance) {
        this(accountNumber, balance, "", "");
    }

    public void depositMoney(double amount) {
        balance += amount;
        System.out.println("Deposited $" + amount + ". New balance: $" + balance);
    }

    public void withdrawMoney(double amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println("Withdrawn $" + amount + ". New balance: $" + balance);
        } else {
            System.out.println("Insufficient funds for withdrawal of $" + amount);
        }
    }
}
```